EliteRestaurant Technical Documentation

This document describes the codebase, runtime structure, architecture, database behavior, API layer, web portal, desktop application logic, connectivity model, configuration system, operational practices, and security posture of the `EliteRestaurant` project.

Repository: `C:\Users\ernes\Documents\EliteRestaurant`
Documentation date: 2026-04-15
Main applications: WPF desktop application, ASP.NET Core API, embedded server portal, seeding and utility tools

Table of Contents

1. Executive Summary

`EliteRestaurant` is a multi-surface restaurant management system implemented primarily in .NET 8. The project is centered around a shared business and data layer contained in `EliteRestaurantPro`. That shared layer is consumed by both a Windows Presentation Foundation desktop application and an ASP.NET Core API. The API also hosts a browser-based server portal inside `wwwroot/index.html`, which makes the system usable from tablets and other LAN-connected devices without introducing a separate frontend framework or a separate frontend repository.

The overall architecture is pragmatic rather than heavily layered. Entity Framework Core is used directly from the shared project, with PostgreSQL as the active database target. The codebase still carries visible legacy traces of an earlier SQLite era, mainly in import and disabled compatibility code paths. The system therefore behaves like a hybrid project that has migrated from a local single-machine database model to a shared network database model while preserving some backwards-compatible logic.

The application domain is broad. It covers menu management, tables, order creation, kitchen preparation flow, cashier and payment flow, reservations, waitlist, attendance, payroll, money tracking, inventory linkage, branding, and reporting. Rather than splitting these features into multiple services, the codebase keeps most of the domain logic in one shared application layer and exposes selected capabilities through the API for remote staff use.

2. Repository and Project Structure

The repository does not currently use a top-level Visual Studio solution file. Instead, the codebase is composed of four separate project files, each built independently. The effective structure is simple and understandable once the shared dependency model is recognized.

Project	Path	Type	Responsibility
`EliteRestaurantPro`	`EliteRestaurantPro/EliteRestaurantPro.csproj`	WPF desktop app + shared core	Main desktop application, shared entities, EF Core context, business utilities, payroll, order logic, settings, exports.
`EliteRestaurant.Api`	`EliteRestaurant.Api/EliteRestaurant.Api.csproj`	ASP.NET Core Web API	REST endpoints, Swagger, static portal hosting, LAN access point for server workflows,

Project	Path	Type	Responsibility
			shared DB access via project reference.
`SeedRunner`	`SeedRunner/SeedRunner.csproj`	Console utility	Reset and reseed PostgreSQL data, generate realistic operational records, optionally reduce active orders.
`ClearActiveOrders`	`Tools/ClearActiveOrders/ClearActiveOrders.csproj`	Console utility	Small operational cleanup helper for active orders; appears to retain stale assumptions from earlier database design.

The repository root also contains `.vscode/tasks.json`, which currently builds and publishes only the WPF project. This is an important operational detail because developers or operators may incorrectly assume that the API is covered by the same task shortcuts when it is not.

Important folders

Folder	Meaning
`EliteRestaurantPro/ViewModels`	MVVM application logic for desktop screens including admin, orders, kitchen, payroll, attendance, inventory, reports, and settings.
`EliteRestaurantPro/Views`	WPF views for the desktop application.
`EliteRestaurantPro/Models`	Entity and domain classes persisted by EF Core or used in business workflows.
`EliteRestaurantPro/Data`	Entity Framework context and data helpers such as financial and query support.
`EliteRestaurantPro/Utils`	Cross-cutting utilities: settings, currency, order workflow, payroll calculations, draft sharing, theming, alerts, exports.

Folder	Meaning
`EliteRestaurant.Api/Controllers`	HTTP entry points for authentication, tables, reservations, health, and server portal behavior.
`EliteRestaurant.Api/Security`	Bearer token parsing, in-memory session state, and portal role validation logic.
`EliteRestaurant.Api/wwwroot`	Single-file browser UI for the server portal.

3. Architectural Overview

Architecturally, the project follows a shared-core model. The desktop application and the API do not maintain separate domain models. Instead, both rely on the same `AppDbContext`, the same entity classes, and many of the same utility classes from the `EliteRestaurantPro` project. This makes the architecture compact and reduces duplication, but it also means the API is coupled to desktop-centric implementation decisions.

High-level runtime diagram

Layer	Implementation	Primary responsibility
User interfaces	WPF desktop UI and browser portal UI	Provide operator workflows for admin, kitchen, cashier, reservations, and server order-taking.
Application logic	ViewModels, utilities, controller methods, workflow helpers	Enforce order states, payroll formulas, table occupancy, financial postings, settings behavior, and session-based features.
Persistence layer	`AppDbContext` with Entity Framework Core	Read and write restaurant data to PostgreSQL, while preserving legacy import support from SQLite.
Data store	PostgreSQL	Shared authoritative backend for all current runtime surfaces.

The absence of a separate repository or service abstraction layer means much of the business logic is close to the UI or controller layer. That choice keeps the implementation approachable for a single-project team, but it also requires discipline when changing data rules because database access can be triggered from multiple places. In practical terms, the project behaves like a desktop-first application that has gradually evolved into a shared local-network system.

Architectural characteristics

- Shared domain model between desktop and API, minimizing duplication.
- MVVM pattern on desktop, with direct EF Core usage from ViewModels and utilities.
- Thin API surface focused on server operations and lightweight health or table queries.
- Static web portal instead of a SPA framework, reducing deployment complexity.
- Settings stored outside the repository in the host machine's local application data.
- PostgreSQL-first runtime with legacy SQLite migration code still present.

4. Desktop Application Architecture

The desktop application is the broadest functional surface in the codebase. It is implemented in WPF on .NET 8 with an MVVM structure. The application starts in `App.xaml.cs`, configures Npgsql compatibility behavior, sets up crash logging, optionally runs a SQLite import mode, validates that a PostgreSQL connection string is available, and then initializes the database schema.

Startup responsibilities

Stage	Observed behavior
Unhandled exception handling	Crashes are logged to `%LocalAppData%\EliteRestaurantPro\logs\app-crash.log` and surfaced to the user.
PDF license setup	QuestPDF is initialized in community license mode.
Legacy import mode	If `--import-sqlite-now` is passed, the app attempts a one-time import from the legacy SQLite database into PostgreSQL.
Database configuration	If no PostgreSQL connection string exists in settings, the user is prompted interactively.
Database initialization	`AppDbContext.Initialize()` runs schema creation and schema patch logic.
Theming	Saved appearance settings are applied before normal UI startup continues.

Desktop feature modules

The desktop system is functionally rich and covers most restaurant operations. Major ViewModel areas include: role selection and login, executive dashboard and drilldown, employee management, menu management, table management, reservations and waitlist, order creation, admin order supervision, kitchen queue, server pickup, money management, salary and attendance, inventory, reporting, and appearance settings.

A notable structural choice is the use of an `AdminBaseViewModel` that centralizes navigation, branding, and session-aware behavior for both full-admin and limited staff tablet experiences. The code therefore uses one application shell for multiple operational personas instead of maintaining separate desktop clients.

Session model

The desktop application uses `AppSession` to track whether the active session is a full admin login or a role-limited tablet-style session such as server, cashier, or kitchen/bar. The session model holds the current employee id, display name, and optional profile image path. Navigation visibility is then driven from those flags, allowing the same application to expose different menus without changing the deployed binary.

5. API and Web Portal Architecture

The API is intentionally small and focused. It is configured in `EliteRestaurant.Api/Program.cs` with controllers, OpenAPI/Swagger, a singleton tablet authentication service, permissive CORS under the policy name `LanOnly`, and static file hosting. The API also calls `AppDbContext.Initialize()` during startup, meaning the API process itself is capable of bootstrapping the shared database schema.

API responsibilities

- Authenticate staff into portal-specific sessions.
- Serve server portal configuration and branding assets.
- Return a server's assigned tables and available menu items.
- Persist and retrieve shared order drafts for the server portal.
- Create or append orders from the web portal.
- Show ready orders and allow marking them as served.
- Expose reservation arrivals and health endpoints.

Server portal frontend

The server portal lives entirely in one file, `EliteRestaurant.Api/wwwroot/index.html`. It contains HTML, CSS, and JavaScript inline. The UI is effectively a hand-written single-page application, but without a framework. It maintains in-memory state for the bearer token, current employee, tables, products, ready orders, shared drafts, configuration data, and the current cart.

This design keeps deployment simple because the API serves both the JSON endpoints and the UI itself. A tablet only needs browser access to the API host over the LAN. The tradeoff is maintainability: the portal file mixes presentation, data fetching, state management, and workflow logic in one place.

API endpoint groups

Controller	Route base	Main purpose
`AuthController`	`/api/auth`	Staff login using staff id plus PIN for a specific portal type.
`ServerPortalController`	`/api/server`	Server-specific configuration, branding assets, products, drafts, orders, ready queue, and serve actions.
`TablesController`	`/api/tables`	Returns available tables for the current session, filtering to assigned server tables for server users.
`ReservationsController`	`/api/reservations`	Exposes arrived reservations.
`HealthController`	`/api/health`	Basic process and database connectivity health checks.

Authentication model

Authentication is implemented by `TabletAuthService`. It is not JWT-based and does not use ASP.NET authentication middleware. Instead, the service creates a random GUID-like token, stores it in a singleton `ConcurrentDictionary`, and returns that token to the client. Subsequent requests send the token in the `Authorization: Bearer ...` header. Validation checks token existence and expiration.

Sessions expire after twelve hours. Because the token store is in memory, all active sessions are lost if the API process restarts. This is acceptable for a lightweight LAN tool, but it is important operationally because tablets will appear logged out after every API restart.

6. Database Design and Persistence Strategy

`AppDbContext` is the single authoritative persistence context. It exposes entity sets for employees, products, tables, orders, order items, inventory items, product ingredients, attendance, payroll artifacts, money transactions, customers, reservations, waitlist entries, and shared order drafts.

On configuration, the context attempts to resolve a PostgreSQL connection string either from environment variables (`ELITE_DB_PROVIDER` and `ELITE_POSTGRES_CONNECTION`) or from the settings file under `Database.PostgreSqlConnectionString`. If no connection string is found, the application throws rather than silently falling back.

Schema strategy

The codebase does not currently use Entity Framework migrations. Instead, initialization relies on `Database.EnsureCreated()` plus additional explicit SQL executed through methods such as `EnsureReservationsSchema`, `EnsureOrderReservationColumns`, and `EnsureSharedDraftSchema`. This approach is fast and convenient for a small project, but it also means schema evolution is manual and embedded in application startup logic rather than tracked in migration files.

Main persisted entities

Entity	Meaning	Notable fields
`Employee`	Staff record for admin, kitchen, cashier, server, and other roles.	`UniqueId`, `SignInId`, `PinCode`, hourly rate, shifts, profile image path.
`Table`	Restaurant floor table definition.	`TableNumber`, name, capacity, status, assigned server.
`Product`	Menu item.	Unique id, category, subcategory, price, ingredient links.
`OrderRecord`	Main sales document / open check / kitchen flow record.	Status, notes, discount, payment fields, order source, reservation reference, timestamps.
`OrderItem`	Line item inside an order.	Product, quantity, assigned prep employee or role label.
`ReservationBooking`	Future or current reservation.	Reserved time, party size, status, deposit fields, linked customer and optional table.

Entity	Meaning	Notable fields
`SharedOrderDraft`	Shared backend snapshot used by server drafts.	Employee ownership, portal, label, JSON payload, timestamps.
`MoneyTransaction`	Ledger-style revenue or expense entry.	Amount in base and converted values, category, date, fixed/variable marker, justification.

The model also defines several useful indexes and uniqueness constraints, for example unique employee sign-in ids when non-empty, unique table numbers, and unique entity `UniqueId` values. Relationships are mostly configured with delete behaviors such as `SetNull` for optional table or server links and `Cascade` for payroll-linked records.

7. Core Business Logic and Workflow Rules

Order lifecycle

The project has a clearly defined order status model in `OrderWorkflow`. The key states are `Pending cashier`, `Waiting`, `In Kitchen`, `Ready`, `Served`, `Completed`, and `Cancelled`. Not all states appear in the same screens, but together they define the operational path.

Status	Meaning	Operational consequence
`Pending cashier`	Server has sent the order but it has not yet entered kitchen flow.	Still treated as an open check and still occupies the table.
`Waiting`	Kitchen should receive the order.	Included in kitchen queue and active order tracking.
`In Kitchen`	Preparation is underway.	Active check remains open and table remains occupied.
`Ready`	Prepared items are awaiting pickup by server.	Visible in the server pickup flow and alert banners.
`Served`	Food has reached the guest and cashier may complete payment.	Still considered open until payment completion.
`Completed`	Order is finalized and financially posted.	Revenue ledger entries can be created and table may become available.

Open check model

A major rule in the codebase is that a table may have one active open check that can continue receiving additional lines until it is completed or cancelled. The API explicitly checks for open checks through the EF-translatable helper `WhereOpenCheckForTable`. The server portal allows the user either to append to the open check or force creation of a separate one, but the code still consistently treats certain statuses as occupying the table.

Shared draft logic

Shared order drafts are stored in the database and keyed to employee id plus portal type. This allows the same employee to save and load drafts across sessions and devices as long as they authenticate with the same staff identity. Drafts are stored as raw JSON snapshots rather than normalized relational data. That choice makes snapshot persistence flexible and decoupled from every small UI state change, at the cost of making drafts opaque to database queries.

Payroll and finance logic

Payroll uses attendance-based deductions and a sales-based bonus. `PayrollCalculator` computes scheduled hours from shift definitions, counts absences and late penalties, computes a five percent bonus on generated sales, and returns a net value that never falls below zero.

`FinancialTransactionService` persists both revenue and salary-related expense entries into the money ledger, linking them back to orders or payroll periods through structured justification text.

Inventory linkage

Products are linked to inventory items through `ProductIngredient`. The seeding utility uses these links to reduce stock when generating historical orders, demonstrating the intended relationship between the menu and stock usage. This implies that the data model is designed for inventory-aware restaurant operations rather than a purely sales-only system.

8. Configuration, Settings, and Branding

Application settings are stored outside the repository in `%LocalAppData%\EliteRestaurantPro\settings\app-settings.json`. The central abstraction is `SettingsManager`, which reads and writes a JSON representation of `AppSettings`.

Settings group	Purpose
`BusinessProfile`	Restaurant name, phone, address, social links, logo path, homepage image, footer text, and legal/tax identifiers.
`CurrencyPricing`	Default display mode, USD to FC exchange rate, rounding behavior, tax percent, and service percent.
`NavigationBackgrounds`	Per-page image paths plus dim and contrast values for navigation screens.
`Database`	Provider marker and PostgreSQL connection string.

The server portal consumes this same settings model indirectly through `/api/server/config`. As a result, branding in the browser is not duplicated in a separate web config file. The restaurant name, logo, tax percentage, service percentage, and currency mode are all derived from the same persisted settings that the desktop app uses. Employee profile photos are resolved from the `Employees` table and served as local files by the API host.

9. Connectivity and Deployment Model

The deployment model is LAN-oriented. The API can be launched on the host machine and bound to a LAN-accessible address, allowing tablets or phones on the same network to open the browser portal. The default development launch settings include `http://localhost:5223` and `https://localhost:7194`, but real device access depends on running with a reachable host binding such as `http://0.0.0.0:5223` and ensuring that local firewall rules allow inbound traffic.

This architecture is especially suited for a restaurant where the main workstation is on Ethernet and staff tablets are on Wi-Fi. No public hosting is required as long as the API host and devices share network visibility. The browser portal is downloaded from the API, and all data exchange occurs through local HTTP requests.

Operational caution: connectivity success depends on correct protocol and port usage. A common field failure is trying to open an HTTPS URL from a device when only HTTP is actually exposed or trusted on the LAN.

10. Security Model and Current Risks

The codebase includes meaningful access separation, but it should be described as pragmatic application security rather than enterprise-grade security. Authentication is PIN-based and role-sensitive. A login request can only succeed if the employee matches the requested portal type, for example server or cashier. That is a positive control because the same employee credentials are not automatically valid for every portal.

Security strengths currently present

- Staff must match both identity and PIN.
- Portal role resolution filters staff by allowed role before creating a session.
- Bearer tokens expire after a fixed duration.
- Server endpoints verify that the caller is a server and that selected tables belong to that server.
- Image asset access uses a token query parameter, preventing anonymous direct reads.

Security limitations and risks

- Sessions are stored only in memory and are not cryptographically signed tokens.
- Pins appear to be stored directly on employee records rather than hashed values.
- CORS policy `AllowAnyOrigin` is intentionally permissive.
- There is no HTTPS enforcement inside the API configuration.
- The project does not use ASP.NET Core authentication middleware, authorization policies, or claims.
- Settings and asset file paths rely on local file-system trust on the host machine.
- The API and desktop both execute schema creation logic at startup, which increases startup-side privileges.

None of these observations mean the system is unsafe for a small local restaurant LAN, but they do define the current trust model: the environment is expected to be controlled, local, and operator-managed. If the project were expanded to internet deployment, multi-site use, or untrusted device networks, the security architecture would need substantial strengthening.

11. Operational Tooling and Maintenance

The repository includes two operational console tools. `SeedRunner` is the primary maintenance tool and is capable of resetting the database, reseeding staff, menu, tables, inventory, and two months of historical activity, and even reducing active orders while preserving business consistency. It is effectively a data reset and demo-data generation utility.

`ClearActiveOrders` appears intended to remove in-progress orders. However, it references a `DatabasePath` property that is not part of the active PostgreSQL-focused code path shown in the current context. This suggests that the utility may contain stale code left over from the SQLite period and should be revalidated before operational reliance.

Observed maintenance model

- Schema changes are embedded in application code rather than migrations.
- The desktop app can prompt for missing database configuration interactively.

- The API can be used as both service endpoint and static portal host.
- Financial consistency is partially repaired through startup helper methods and reconciliation routines.
- Table state consistency can be recalculated from active orders using `ReconcileTableStatusesWithOrders`.

12. Architectural Strengths, Constraints, and Recommendations

Strengths

The codebase is cohesive and direct. Desktop and API share the same business understanding, the operational workflows are easy to trace, and the system is very suitable for a single-site restaurant. The browser portal is deployment-light, the settings model is unified, and the domain coverage is broad enough to function as a real restaurant management platform rather than a narrow point-of-sale sample.

Constraints

The same cohesion that makes the project approachable also increases coupling. Because the API depends on the desktop project, web-facing logic inherits many assumptions from the WPF application. Database evolution is manual. Security controls are lightweight. The portal frontend is concentrated in one HTML file. Some helper utilities still reveal legacy SQLite residue. These are not immediate blockers, but they are the main scaling constraints.

Recommended next improvements

- Introduce formal EF Core migrations for repeatable schema evolution.
- Split `wwwroot/index.html` into separate HTML, CSS, and JavaScript files for maintainability.
- Move authentication to signed tokens or middleware-backed auth if broader deployment is planned.
- Hash staff PINs instead of storing raw values.
- Tighten CORS and prefer HTTPS for device access where operationally feasible.
- Revalidate or modernize `ClearActiveOrders` against the PostgreSQL-first design.
- Consider separating shared domain logic from WPF-specific concerns into a dedicated class library.

This document is intentionally architecture-first. A deeper appendix with entity detail, endpoint reference, folder inventory, operational notes, and technical observations is provided separately in `EliteRestaurant-Technical-Appendix`.